Eber Felipe Barrotti Louback

Laboratório 02:
Manipulação de processos

Relatório técnico de atividade prática solicitado pelo professor Rodrigo Campiolo na disciplina de Redes de Computadores II do Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR

Departamento Acadêmico de Computação – DACOM

Bacharelado em Ciência da Computação – BCC

Campo Mourão

Março / 2026

Resumo

Este trabalho busca experimentar com processos de sistema operacional. Foram executados comandos como `ps aux`, tendo seus resultados analisados. Além deles, também foram criados programas que manipulam e executam processos, usando funções como o `fork()`. Todos os comandos e programas tiveram os seus objetivos alcançados.

Palavras-chave: processos. sistemas operacionais.

Sumário

1 Introdução 4
2 Objetivos 4
3 Materiais 4
4 Procedimentos e Resultados 4
4.1 Manipulação de Processos 4
4.1.1 Execute o comando ps aux e identifique três processos do sistema (daemons) e três processos interativos. Explique os valores da saída desse comando para um dos processos identificados. 4
4.1.2 Há processos zombies executando em seu sistema operacional? Posso eliminá-los do sistema usando o comando kill -SIGKILL pid_zombie? Justifique. 6
4.1.3 Quais os processos com maior utilização de CPU? Quais os processos com maior utilização de memória? Qual o processo do usuário está a mais tempo em execução? . . . 6
4.1.4 Como eu faço para suspender um processo no Linux? Como eu faço para retomar a execução novamente? 7
4.1.5 O que aconteceria se um processo criasse recursivamente processos filhos indefinidamente? Implemente um programa em Linux que faça isso e apresente o resultado. (Sugestão: testar na máquina virtual). 8
5 Discussão dos Resultados 9
6 Conclusões 9

1 Introdução

Processos e seu gerenciamento são partes centrais do funcionamento de um sistema operacional. Este trabalho explora comandos e programas que os monitoram e manipulam.

2 Objetivos

- Compreender a estrutura de processos em Linux.
- Manipular processos em linguagem de programação.

3 Materiais

- Distribuição Linux: Arch 6.19.6-arch1-1.
- Ambiente de desenvolvimento: Naovim.
- Comandos do sistema e bibliotecas de programação:
  - `unistd.h`: Biblioteca que dá acesso a API POSIX. Inclui os comandos `fork()`, `execl()` e `execv()`.
  - `sys/wait.h`: Inclui o comando `wait()`.
  - `sys/types.h`: Tipo `pid_t`.
  - `string.h`: Métodos que lidam com *strings* como: `strcat()`, `strcspn()` e `strlen()`.
  - `time.h`: Biblioteca para manipular tempo, `time()` foi usado para aleatorizar vetores no programa `busca_vetor.c`.

4 Procedimentos e Resultados

4.1 Manipulação de Processos

4.1.1 Execute o comando ps aux e identifique três processos do sistema (daemons) e três processos interativos. Explique os valores da saída desse comando para um dos processos identificados.

- Interativos:

Figura 1 – Chromium

- PID: Id do processo, 66191
- %CPU: Porcentagem do tempo do processo na CPU, 9.2%
- %MEM: Porcentagem de memória RAM utilizada, 3.1%
- VSZ: Tamanho da memória virtual, em KB, 51396476
- RSS: Quantidade de memória física ocupada pelo processo, também em KB, 217844
- TTY: Interface de terminal, `?` indica que o processo não está atrelado a um
- STAT: Estado do processo, S indica que está esperando evento, s indica que é líder de sessão e l indica que é *multi-threaded*
- START: Hora em o processo foi inicializado, 14:03
- TIME: Tempo total de processamento acumulado, 2:01
- COMMAND: Comando que iniciou o processo e todos os seus argumentos. Para facolitar a vizualização, eles foram cortados das imagens, nem mesmo no terminal é possível vê-los completamente. `/usr/lib/chromium/chromium -ozone-platform=wayland -ozone-platform-hint=wayland -enable-features=Touc -load-extension= /.local/share/omarchy/default/chromium/exten`

Figura 2 – Steam

Figura 3 – Spotify

[!h]

- *Daemons*:

Figura 4 – *Daemon* do Avahi

Figura 5 – *Daemon* que muda a política de agendamento de processos/*threads*

```
ener       1706  0.0  0.1  34424  8640 ?        Ss   14:11   0:00 /usr/bin/python3 /usr/bin/uwsm aux app-daemon
```

Figura 6 – Daemon do python

4.1.2 Há processos zombies executando em seu sistema operacional? Posso eliminá-los do sistema usando o comando kill -SIGKILL pid_zombie? Justifique.

Sim, existem processos *zombies* executando no meu sistema, como é possível ver na Figura 7.

```
~ ✗ ps aux | grep 'Z'
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ener        1620  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
ener        1621  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
ener        1627  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
ener        1631  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
ener        1635  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
ener        1639  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
```

Figura 7 – Processos zombies encontrados com o comando `ps aux | grep 'Z'`

Não, matar um zumbi com o comando `kill -SIGKILL pid` não funciona, como evidenciado na Figura 8. Para terminar um zumbi, é necessário matar o seu processo pai.

```
~ ❯ kill -SIGKILL 1620

~ ❯ ps aux | grep 'Z'
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ener        1620  0.0  0.0      0     0 ?        Z    08:45   0:00 [uwsm-app] <defunct>
```

Figura 8 – Tentativa de matar um processo zumbi com o comando `kill`. Ainda poder com um `ps aux` mostra como isso não funcionou.

4.1.3 Quais os processos com maior utilização de CPU? Quais os processos com maior utilização de memória? Qual o processo do usuário está a mais tempo em execução?

Os processos que mais utilizaram CPU e memória e que executaram por mais tempo são todas o Chromium, como visível nas figuras de 9 a 11, respectivamente.

```
~ ❯ ps aux --sort=-pcpu
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ener       85930  9.3  3.7 1459894932 259280 ?   Sl   14:53   0:03 /usr/lib/chromium/chromium
ener       66191  8.8  5.4 51421144 378352 ?     Ssl  14:03   4:25 /usr/lib/chromium/chromium
```

Figura 9 – Programas com maior %CPU

```
~ ❯ ps aux --sort=-pmem
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ener       66191  8.7  5.4 51421144 376140 ?     Ssl  14:03   4:29 /usr/lib/chromium/chromium --
ener       74063  5.4  4.9 1459904376 340792 ?   Sl   14:22   1:47 /usr/lib/chromium/chromium --
ener       66624  4.2  4.7 1465089332 328176 ?   Sl   14:04   2:09 /usr/lib/chromium/chromium --
ener       85930  3.4  3.2 1459903000 224696 ?   Sl   14:53   0:03 /usr/lib/chromium/chromium --
```

Figura 10 – Programas com maior %MEM

```
~ ❯ ps aux --sort=-time
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ener       23855 22.6  6.3 1466773468 437976 ?   Sl   18:54  10:21 /usr/lib/chromium/chromium
ener       23554  9.2  5.1 51347700 354048 ?     Ssl  18:54   4:14 /usr/lib/chromium/chromium
ener        1527  1.0  0.8 2479820 58128 ?       Sl   14:11   3:21 Hyprland --watchdog-fd 4
ener       23650  6.5  3.4 51304804 241228 ?     Sl   18:54   2:59 /usr/lib/chromium/chromium
ener       21545  3.0  1.0 372828 73480 ?        Rsl  18:48   1:34 nvim --embed .
```

Figura 11 – Programas com maior TIME

4.1.4 Como eu faço para suspender um processo no Linux? Como eu faço para retomar a execução novamente?

Para pausar um processo no Linux se pode usar o comando: `kill -STOP <pid>` e para retomar o processo basta usar: `kill - CONT <pid>`. Esses passos podem ser observados nas Figuras de 12 a 16.

```
ener       88267  0.0  0.0   7848  4676 pts/1    S+   14:59   0:00 ping 8.8.8.8
```

Figura 12 – Processo de `ping` com pid 88267

```
~ ❯ kill -STOP 88267
```

Figura 13 – Comando que pausa a execução

```
64 bytes from 8.8.8.8: icmp_seq=36 ttl=116 time=24.5 ms

[1]+  Stopped                 ping 8.8.8.8
```

Figura 14 – Execução do ping pausada pelo comando

Figura 15 – Comando para retomar a execução

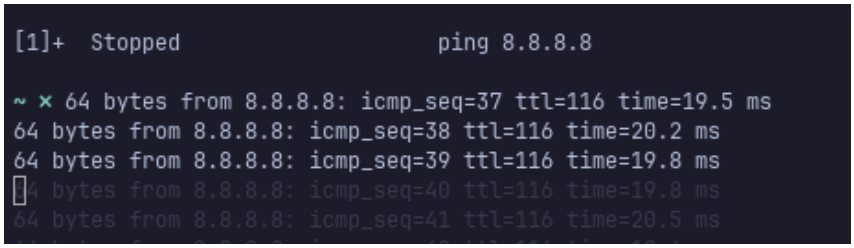

Figura 16 – Comando `ping` retomado

4.1.5 O que aconteceria se um processo criasse recursivamente processos filhos indefinidamente? Implemente um programa em Linux que faça isso e apresente o resultado. (Sugestão: testar na máquina virtual).

Ao testar um programa que cria filhos indefinidamente, a máquina virtual começou a ficar muito lenta e outros processos começaram a congelar.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

/*
 * Autor: Eber Felipe Barrotti Louback
 * Descrição: Criar filhos recursivamente, infinitamente
 * Data de criação: 30/03/2026
 * Data de modificação: 30/03/2026
*/


void recursiveForks() {
  fork();
  recursiveForks();
}

int main() {
  recursiveForks();
  exit(0);
```

```
}
```

5 Discussão dos Resultados

Com os resultados alcançados no decorrer das etapas, é possível dizer que os objetivos foram sim alcançados. As atividades propostas permitiram que a estrutura de processos em Linux fosse bem observada e compreendida, fundamentando conceitos como tempo de CPU e tamanho virtual.

Esse conhecimento foi essencial na hora de manipular os processos, tanto usando comandos no terminal, quanto por código, permitindo a superação de dificuldades na hora de criar uma hierarquia de processos, por exemplo, que foi a parte mais complicada de realizar das duas partes da atividade.

6 Conclusões

A atividade teve uma conclusão satisfatória, com todos os passos sendo completados de forma aceitável. O trabalho proporcionou um aprofundamento considerável no tema e ensinou vários conceitos importantes para um tópico fundamental na Computação.